1. Hafta

! Time (zaman) bize ne kadar CPU kullandığımızı söyler. Time için karmaşıklığımız 2 koda aynı değeri girip ne kadar sürdüğüne bakarız.

~ Algoritma Analizi ~

Fibonnaci algoritması gördük. Recursive fonksiyonu ve bunu recursive değil de for döngüsüyle yapılacak şekilde karşılaştırdık Python'da.

Elimizde bir problem var → Çözüm 1
Hangi çözümü kullanmalıyız? Çözüm 2
Çözümleri karşılaştırma kaynak kullanımı, çözüm kolaylığı gibi şeylere bakılabilir. ⋮

* Algoritma için problem nedir?

Kriterler

- time → CPU, RAM, source, time → zamanına bakabiliriz. } Time en kritik olan.
- easy to use (code) } kolay olanı hatasız (−) yazmak daha kolay
- memory → kapladığı bellek alanı (−) } önemli ama zaten veri elimize sığıyor olmalı. (öyle düşünüyoruz)

ignore → hardware, OS, IDE, PL (C#, Python...)
↳ Bunlara bakmadan karmaşıklığı ölçüyoruz. (görmezden geliyoruz.)

Fibonnaci Algoritması için

* 1. yöntemde direkt pc ortamda zamanın gösterilmesi.

	recursive	loop
n=35	0.7 ↳ n=43'te 36 sn.	0.0
n=36	1.1	0.0
n=40	8.2	0.0
n=42	21.9	0.0
n=50	$21.9 \times 2^{10} \cong 8000$ sn.	0.0

} Rewrsive'de her bir adımda 2'ye katlanıyor
n = 50 → 1 saat
n = 51 → 2 saat
n = 60 → 1024 saat → 40 gün

karmaşıklığı bu şekilde seri şeklinde tahmin edebiliriz.

! 2. yöntemde yazdığımız kodda zamanı (time) ölçmek için algonun çalışması sırasında kaç komut devreye giriyor. Buna göre zamanı tahmin ediyoruz.

! Algoritmanın karmaşıklığı genelde en kötü olandan ilerleyir.

* Mark Allen Weiss 'in algoritma kitabından işledik. (chapter 2)
↳ Data Structures and Algorithm C++

Recursive'de komutların tamamına T zaman da dersek;

$$T(n) \xrightarrow{\text{n sayısı için}} \overline{T(n-1)}^{\,1} + T(n-1)$$

$T(n) = T(n-1) + T(n-2) + 1 \rightarrow T(n) = 2^n \cdot T(1)$
$T(n) = 2\,T(n-1) + 1$
$T(n-1) = 2 \cdot T(n-2)$
$T(n-2) = 2 \cdot T(n-3)$
$T(2) = 2 \cdot T(1)$

(Recursive Fibonacci)

! Zamanı ölçmek için 2. çözüm → 2^n oluyor → 2^{40} oluyor. ($n=40$ için)
Looplu olan için n oluyor. komut $n=40$ için 40,
Bunların hepsi tek bir fonksiyona denk geliyor.
O da lineer fonksiyon.

! Çözümleri zaman açısından bu şekilde karşılaştırabiliyoruz.
2. çözümde komutları sayarak bir fonksiyon grubu bulduk.
Bunlar time için prediction oluyor.
- n dediğimiz aslında → $an+b$ → lineer karmaşıklık
- 2^n dediğimiz → a^n, $b.a^n + \ldots$ → exponential karmaşıklık

Karmaşıklıklar

polinomial (P) algoritmalar:
- 1 → sabit değer (2, 3 yıl gibi...)
- $\log n$
- n
- $n \cdot \log n$
- n^2
- n^3
- ⋮

NP algolar: 2^n } çözülemeyen uzun süren algolar

Algoritmalar bunlardan birine sahip olabilir. Karmaşıklık NP ise iyi bir yöntem değildir. AI'da heuristic yöntemlerle, greedylerle NP'ler çözülmeye çalışılıyor.

! Karmaşıklıklar için bir fonksiyon grubu olacak.
Algoritmalar için karmaşıklık bulacağız.
Liste, ağaç yapılarında bunlar nasıl ilerliyor gibi...